Cost allocation del servicio `dss`

Propuesta de método para pasar del costo que reporta el proveedor (por cluster) al costo por aplicación — justo, estable en el tiempo y explicable.

2025-04 → 2025-09 | dbt + BigQuery | ~120 entidades | Asignado + no asignado = facturado

ESTADO ACTUAL

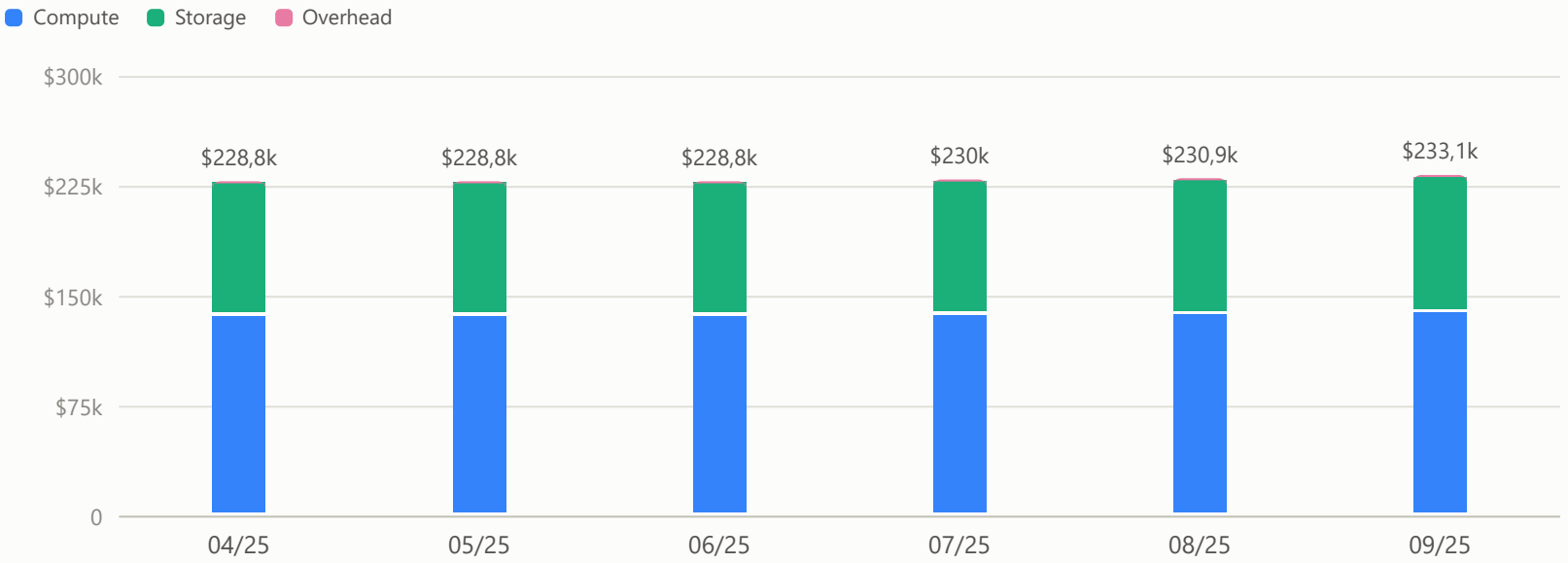
Costo asignado por entidad, por driver, en el tiempo

ENTIDAD Todas (agregado) PERÍODO Completo (abr–sep, prom. mensual)

Facturado	Asignado	Sin asignar	Entidades con costo
$249,2k	**92.3%**	**$115.105**	**120**
/mes · $1.495.450 en 6 meses	$1.380.345 en 6 meses	$19,2k/mes · causa identificada	reparto plano: la mayor pesa 3.0% del total

Costo asignado por driver — Todas las entidades (agregado)

La barra completa es el costo total asignado del mes, por driver.



TOP 5 POR COSTO

app-billing-05	$6,9k
app-buyer-03	$6,3k
app-shipping-04	$6,2k
app-search-04	$5,4k
app-buyer-07	$5,2k

▶ Ver datos en tabla

Fuente: marts `fct_cost_allocated_entity_period` + `fct_cost_unallocated_detail` (+ `fct_unassigned_usage`), reconciliadas contra el billing corregido con tolerancia $0,01 · pipeline dbt + BigQuery, DAG de Airflow incluido. Lo no asignado, en la slide siguiente.

ESTADO ACTUAL

Lo que queda afuera — y qué hacemos con ello



Asignado a entidades **$1.380.345** · Clusters sin mapeo a set **$54.450** · Sets sin entidades vigentes **$60.655**

No asignado — total	Clusters sin mapeo	Sets sin entidades	Uso huérfano
$115.105	**$54.450**	**$60.655**	**5 apps**
7,7% del facturado · $19,2k/mes	4 clusters (cl-001/041/042/114)	3 sets permanentes + cs-058 (jul)	reportan uso, no reciben costo

Costo no asignado por mes (USD)

Reason	Abr	May	Jun	Jul	Ago	Sep
unmapped_cluster (cl-001/041/042/114)	$9.075	$9.075	$9.075	$9.075	$9.075	$9.075
orphan_cluster_set (cs-010/035/040 + cs-058 en jul)	$9.964	$9.964	$9.964	$10.835	$9.964	$9.964

Propuesta: los 4 clusters sin mapeo son deuda de gobernanza del mapa (dueño + fecha, no diluir en un prorrateo general); los 3 sets huérfanos permanentes se resuelven definiendo si son capacidad reservada/entornos internos (se absorben como costo de plataforma) o deben apagarse.

Los montos son casi planos mes a mes porque estos clusters/sets facturan a tarifa diaria estable en el feed y no dependen del uso; el único quiebre es cs-058 en julio. Con precios variables o distinta cantidad de días, fluctuarían.

El salto de julio es cs-058: factura desde el 1/7 pero su única app entra en vigencia el 16/7 (+$871) — hallazgo del pipeline, pregunta abierta en la última slide.

Uso sin costo (evidencia)

Reason	Volumen	Entidades
orphan_entity_usage — compute (app-buyer-01, app-support-01, app-returns-02, app-search-05, app-loyalty-04)	1.429.230 vCPU-h	5
orphan_entity_usage — storage	2.364.543 GB	5
excluded_outlier_usage — storage (app-reporting-01, 15/7)	45.000 GB	1

El overhead sin tag (DataTransfer/CloudWatch/GuardDuty/Support, $6,4k) **no** está en la tabla de arriba: se prorratea y sí se asigna como driver `overhead` (ver slide siguiente).

MÉTODO

Del costo por cluster al costo por entidad

Costo del proveedor por cluster → corrección de datos → costo por **cluster_set** → costo por **entidad**, prorrateado por su uso diario del driver correspondiente. EC2 se reparte por vCPU-hours y S3 por GB — nunca se mezclan unidades.

1
Billing por cluster
`cluster_billing_lines` — día × cluster × usage_type, cuenta AWS

→

2
Cluster → cluster_set
`cluster_set_membership` — pool lógico de clusters

→

3
Set → entidad (vigencia)
`entity_clusterset_map` — valid_from/valid_to por app

→

4
Prorrateo por uso
vCPU-h (compute) / GB (storage) diario de cada entidad vigente

→

5
Costo por entidad
`fct_cost_allocated_entity_period` — billing_period × entity × driver

Corrección cuenta 846 (paso 1): dato versionado en una seed (÷1000, con vigencia y autoría), nunca hardcodeado en el código.

Cluster en 2 sets (paso 2): split proporcional al uso agregado de cada set ese día; 50/50 solo si ninguno tiene uso.

Overlap de vigencia (paso 3): si una entidad tiene 2 sets vigentes a la vez (app-fraud-04), ambas cuentan — no se elige una.

Overhead sin tag (no tiene cluster): DataTransfer/CloudWatch/GuardDuty/Support ($6,4k) se prorratea por costo directo de cada entidad y se asigna como driver `overhead` propio — no queda sin asignar.

Un test automático (`assert_reconciliation`) garantiza en cada corrida: asignado (compute+storage+overhead) + no asignado = facturado corregido, al centavo.

DECISIONES

Decisiones de método y sus trade-offs

1

Corrección 846 como dato versionado

Costo unitario implícito $59/vCPU-h vs $0,03–0,08 en el resto; corregida (÷1000), el total cae en la referencia de negocio ($249k/mes). Es el número más grande del servicio: remueve **~$1,5M/mes** de la cuenta 846 (de $1,5M reportado a $1.500). Vive en una seed con vigencia y razón.

Trade-off: hasta validación del dueño es una hipótesis (`pending_owner_validation`); la reconciliación "al centavo" corre sobre el feed *ya corregido*.

2

Prorrateo por uso medido, por driver

Compute por vCPU-hours, storage por GB, por día: justo (paga quien usa), estable frente a picos propios (el uso diario los amortigua) y explicable (cada dólar rastreable a una fila).

Trade-off: depende de la calidad del feed de uso del dueño. La estabilidad es frente al uso *propio*: en pools compartidos, la factura igual puede moverse por el uso de las vecinas (ver decisión 3).

3

Reparto de costo compartido: proporcional al uso

Un cluster en 2 sets se parte por el uso de cada set (50/50 si ninguno usa). Dentro de un set, cada entidad paga por su uso — así, su factura puede moverse por el uso de sus **vecinas**, no solo por el propio. Exposición: **$42.276 en 6 clusters (~2,8% del total)**; el resto se asigna sin ambigüedad de pool.

Trade-off: gana precisión, pierde previsibilidad. Mitigación: descomponer la varianza en "por tu uso" vs. "por composición del pool"; suavizar o fijar una porción queda para el dueño.

4

Solapamiento de vigencias: ambas válidas

app-fraud-04 pesa en dos sets durante el solapamiento (cs-018 + cs-060); elegir uno inventaría una precisión que el mapa no tiene.

Trade-off: durante el solapamiento recibe costo de *dos* pools a la vez — puede pagar más que una entidad equivalente en un solo set. Mitigación: el período de solapamiento es acotado y auditable (fechas de vigencia en el mapa); si el dueño lo considera doble cobro, se resuelve cerrando la vigencia vieja al migrar — es una decisión de mapa, no de método.

5

Pool de 1 entidad con uso cero: se asigna igual

Cuenta 512 factura $1k en septiembre con su única app apagada — se asigna con `sole_member_no_usage`, flageado por un test en cada corrida.

Trade-off: cobra capacidad reservada, no uso — discutible, por eso nunca es silencioso.

6

Outlier de uso: excluido del peso, no corregido

La fila de 45.000 GB de app-reporting-01 (patrón ~2.000) se detecta por mediana móvil y se excluye de la prorrata, visible como evidencia.

Trade-off: reemplazar el valor sería inventar datos — preferible excluir y mostrar.

AGENDA DE VALIDACIÓN

Preguntas abiertas al dueño del servicio

Puntos concretos para la reunión — cada uno con un número accionable detrás.

1 **Cuenta 088420000846 — escala ×1000**

Es la cifra más grande en juego: la corrección remueve **~$1,5M/mes** (de $1,5M reportado a $1.500). ¿Confirmás que reporta costo ×1000 (bug de unidad en el feed)? ¿Desde cuándo? ¿Se corrige en origen o mantenemos la corrección versionada en la seed?

2 **cs-058 — ramp-up de app-payments-02**

Facturó 15 días antes de que su única app tuviera vigencia (1–15 jul, $871). ¿Retro-asignamos ese costo a app-payments-02 o queda como costo de plataforma?

3 **Gobernanza del mapa cluster↔set↔entidad**

¿Quién es dueño de cl-001/041/042/114 (sin set) y de cs-010/035/040 (sin apps)? ¿Son entornos internos, capacidad reservada, o falta de mantenimiento? ¿Y las 5 apps con uso sin mapeo — quién las da de alta, con qué SLA?

4 **Chargeback: directo vs. fully-loaded**

Para cobrar formalmente a cada equipo, ¿usamos costo directo (compute+storage) o fully-loaded (+ overhead prorrateado)? Ambos conviven en la misma tabla, por filtro.

5 **Caso 512 / septiembre — capacidad ociosa**

¿Facturar con uso cero es capacidad reservada deliberada? ¿Corresponde cobrarla igual a la app, como hace hoy la propuesta?

6 **Costo compartido entre entidades del mismo pool**

Hoy el costo de una entidad puede subir aunque no haya cambiado su uso, solo porque una vecina del mismo cluster_set usó menos. ¿Preferís estabilidad (suavizar con promedio móvil o separar una porción fija) o precisión (mantener el prorrateo diario tal cual)?

7 **Base de prorrateo del overhead sin tag**

El overhead ($6,4k, 0,4% del total) se reparte hoy proporcional al costo directo de cada entidad. Es una base razonable pero no la única — podría ser por nº de instancias, servicios activos, o partes iguales. Como pesa tan poco casi no cambia el número, pero ¿confirmás esta base o preferís otra?